

GRAIL-VN: A Graph-Grounded Runtime-Verifiable Agentic Framework for Reliable Branching Visual Novel Script Generation

Chuanru Wei^{1*}, Kaili Chen^{2*}

¹Nankai University, Tianjin, China

²Southeast University, Nanjing, China
weichuanru3@gmail.com, kailichen@seu.edu.cn

Abstract

The core challenge in visual novel script generation lies not only in generating coherent narrative content, but also in continuously maintaining the consistency of branching structures, character states, and conditional logic as the story evolves. This paper proposes GRAIL-VN, a Graph-Grounded Runtime-Verifiable Agentic Framework for Reliable Branching Visual Novel Script Generation, which models visual novel script generation as an iterative update process based on narrative state. The framework maintains persistent narrative state through a Narrative World Graph (NWG), performs state retrieval, local generation, and state commitment through a graph-backed iterative runtime (GIR), and applies multi-layer runtime verification before updates are written back through Plan-Graph-Constrain-Realize (PGCR); Lookahead Branch Planning (LBP) and Just-In-Time Casting (JITC) further support lookahead branch decision-making and dynamic expansion of the narrative world. We conduct experiments on 36 narrative briefs covering different genres and complexity levels, and perform an integrated analysis combining automated structural evaluation, LLM pairwise narrative quality evaluation, and goal achievement evaluation. The results show that the value of the persistent graph mainly comes from its continuous participation as a runtime state interface, rather than from one-time reading during the generation stage; PGCR can effectively detect and repair inconsistent updates, thereby improving generation reliability, while LBP and JITC further enhance branch planning and dynamic expansion capabilities. Overall, GRAIL-VN provides a unified runtime framework for reliable branching visual novel script generation.

Code — <https://github.com/Konjacor/GRAIL-VN>

Introduction

Automatic story generation is an important problem in computational narrative (Alabdulkarim, Li, and Peng 2021), and branching interactive narrative further raises this problem to a higher level of difficulty: a system must not only generate natural-language text, but also maintain executable scene topology, conditional branches, character relationships, and state evolution. Visual novels (VN) are a representative instance of this problem, because their scripts more closely

resemble executable narrative graphs driven by logical state. Their quality depends not only on whether local text is natural and global text is rich, but also on cross-scene consistency and the usability of branching structures.

Although current LLM-based narrative generation methods have made progress in long-form text creation and interactive scripting (Yao et al. 2019; Mirowski et al. 2023), many existing approaches still rely on implicit context accumulation to maintain evolving narrative world state. When a story contains multiple scenes and branches, this mechanism can easily lead to forgotten character attributes, unverifiable conditional branches, and inconsistencies between earlier and later settings. Multi-agent Director-Actor frameworks go further in narrative control (Chen et al. 2024; Han et al. 2024), but many implementations still lack a shared, persistent, and verifiable state foundation, as well as generation-time alignment and constraint mechanisms operating around that state foundation.

More importantly, such errors often continue to propagate as the narrative progresses. Because subsequent scenes depend on prior character states, event outcomes, and conditional variables, an undetected state error may continuously affect future scene generation and branch decisions, ultimately leading to invalid conditions, abnormal character behavior, or unreachable branches. This indicates that relying solely on the context window to implicitly maintain narrative state is insufficient to support long-term consistency in complex branching narratives, and also insufficient to perform system-level runtime control over local narrative updates.

Based on this observation, this paper models visual novel script generation as an iterative generation process driven by explicit narrative state. In this process, the system continuously reads the current narrative state, generates local candidate updates, and performs verification and repair before writing them back, thereby forming a traceable closed loop of state evolution. To implement this process, this paper proposes GRAIL-VN: a visual novel generation framework that maintains persistent narrative state with a Narrative World Graph (NWG), drives stepwise generation and state commitment with a graph-backed iterative runtime (GIR), and performs pre-commit verification and repair with Plan-Graph-Constrain-Realize (PGCR). Compared with generation pipelines that implicitly store state in textual context,

*These authors contributed equally.

GRAIL-VN externalizes narrative state into a structural object that can be queried, traced, and written back, enabling it to continuously participate in generation, verification, and subsequent decision-making as a runtime state interface during generation.

On top of this main closed loop, this paper further introduces Lookahead Branch Planning (LBP) and Just-In-Time Casting (JITC), which are used to enhance branch filtering and dynamic character expansion capabilities, respectively. The experiments combine automated structural evaluation, runtime repair diagnostics, LLM-as-a-Judge evaluation (Zheng et al. 2023), and goal achievement evaluation to systematically analyze the effects of explicit state, runtime verification, and extension mechanisms on structural reliability, state consistency, and narrative quality.

The main contributions of this paper are threefold:

1. We model branching narrative generation as an iterative update process based on explicit narrative state, enabling narrative state to continuously participate in reading, verification, and write-back as a runtime object during generation.
2. We propose the GRAIL-VN framework, which constructs a readable, verifiable, repairable, and committable state-driven generation loop through NWG, GIR, and PGCR, and supports branch filtering and dynamic character expansion on this loop through LBP and JITC.
3. We construct an evaluation system oriented toward runtime analysis, analyzing the operational boundaries of runtime state control mechanisms from the perspectives of structural outcomes, repair trajectories, retry cost, and external narrative quality.

Related Work

Interactive Narrative and Visual Novel Generation

Interactive narrative generation has long sought to balance narrative coherence with open-ended and interactive storytelling (Alabdulkarim, Li, and Peng 2021; Riedl and Young 2010). Early systems were primarily grounded in symbolic planning and goal-driven methods such as TALE-SPIN (Meehan 1977), IPOCL (Riedl and Young 2010), and Sabre (Ware and Siler 2021), which explicitly modeled character goals, action preconditions, and causal dependencies to provide strong formal guarantees of narrative consistency. With the emergence of neural language models and large language models, systems such as Plan-and-Write (Yao et al. 2019), DOC (Yang et al. 2023), and Dramatron (Mirowski et al. 2023) further improved long-form narrative organization through hierarchical planning, outline control, and collaborative scripting. More recently, Director-Actor and multi-agent frameworks, including HoLLMwood (Chen et al. 2024), IBSEN (Han et al. 2024), CoDi (Kim, Yoo, and Cheong 2025), and StoryVerse (Wang, Zhou, and Ledo 2024), have separated high-level narrative planning from local character execution, improving controllability and character agency in complex storytelling. Despite these advances, existing approaches primarily focus on how narrative content is generated and how multiple agents coordinate generation, while giving comparatively less attention to

how continuously evolving narrative state supports complex branching decisions in visual novel generation.

Structured State and Narrative Consistency

To address long-range consistency, recent work has increasingly introduced structured knowledge graphs, external memory, and long-term memory mechanisms. Knowledge-graph-enhanced generation models entity relations and factual knowledge to improve consistency and controllability (Gardent et al. 2017; Koncel-Kedziorski et al. 2019; Guan et al. 2020), while memory-augmented approaches support experience retention and behavioral consistency during long-term interaction. Systems such as Generative Agents (Park et al. 2023) and Voyager (Wang et al. 2023) further demonstrate the value of explicit memory for persistent behavior modeling in dynamic environments. These studies collectively recognize that textual context alone is insufficient for complex narrative state management. However, existing graph and memory modules are primarily designed for retrieval augmentation, background knowledge, or experience recall rather than functioning as explicit runtime state throughout narrative generation. Consequently, character attributes, event preconditions, and branch conditions can be recorded but remain difficult to query, update, and reuse as a unified system state.

Reliable Narrative Generation through Verification

Recent large language model systems have increasingly incorporated verification and self-correction mechanisms, including critique-and-revise and self-reflection, to improve output reliability (Madaan et al. 2023; Shinn et al. 2023). These approaches typically identify generation errors through additional review stages and refine generated content using feedback, thereby improving factual consistency, logical correctness, and task completion quality. Although these methods demonstrate the value of post-generation verification, their verification targets are typically completed textual outputs rather than the continuously evolving narrative state during generation. As a result, verification is generally performed only after content has been generated, making it difficult to directly constrain state evolution or continuously ensure consistency in complex branching narratives.

Overall, prior work has advanced these components largely independently. GRAIL-VN addresses this gap by integrating state management, content generation, and pre-commit verification into a single runtime, treating branching visual novel generation as a sequence of verifiable narrative state transitions.

Method

Framework Overview

We formulate the task as follows. Given a narrative brief B , GRAIL-VN generates a structured visual novel script S together with an evolving Narrative World Graph G . Formally,

$$S, G_T = \mathcal{F}(B; G_0), \quad (1)$$

where $\mathcal{F}$ denotes the overall GRAIL-VN framework, G_0 denotes the initialized narrative state and G_T the final narrative

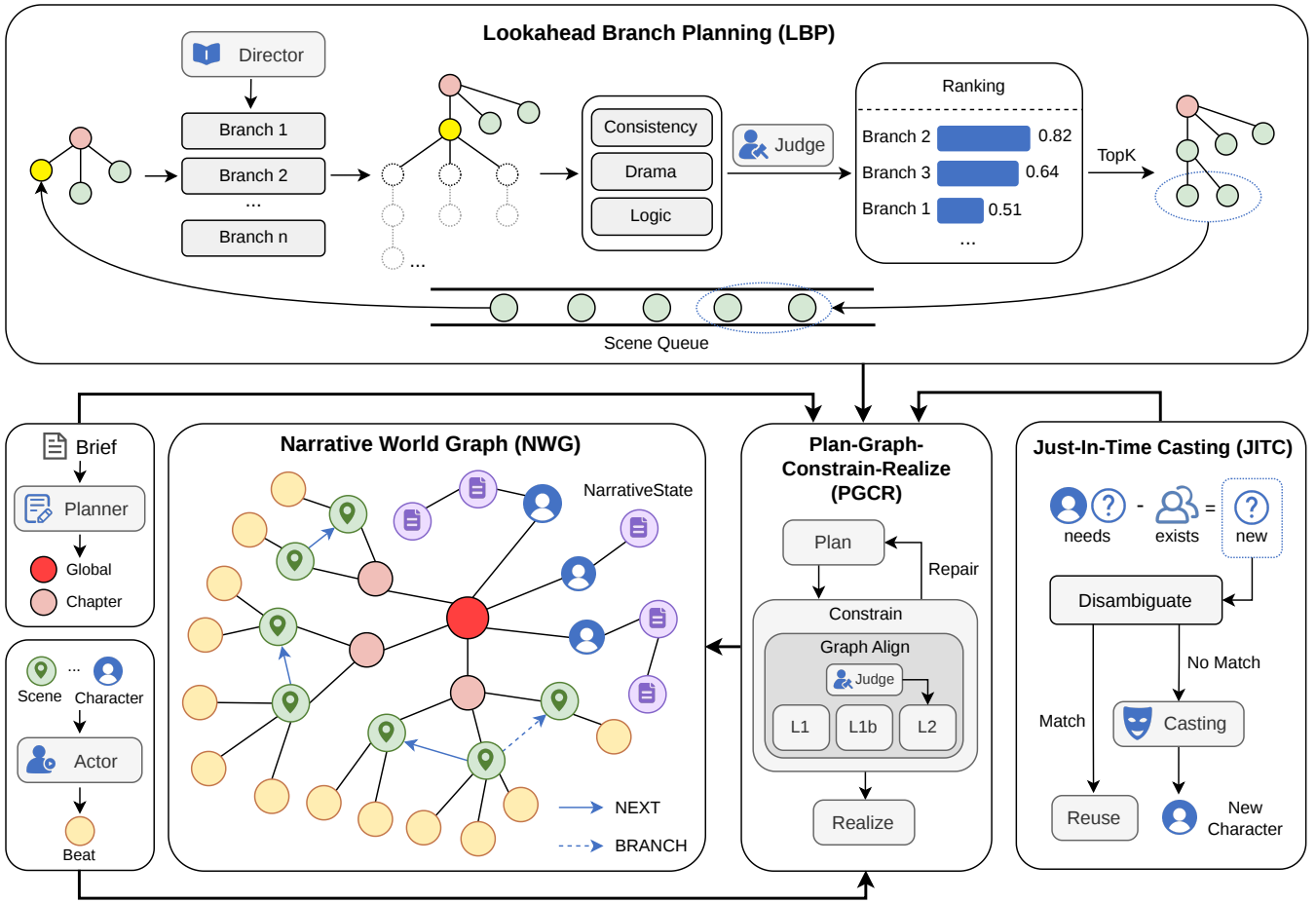


Figure 1: GRAIL-VN Framework Overview.

world graph. Rather than treating generation as a one-shot text synthesis problem, GRAIL-VN formulates branching visual novel generation as a sequence of iterative narrative state transitions. At each step, the system reads the current graph state, generates candidate narrative updates, validates them before commitment, and writes accepted updates back to the graph.

Figure 1 illustrates the overall framework. GRAIL-VN consists of four tightly coupled components. NWG serves as the persistent runtime state by maintaining narrative structures, character entities, and state snapshots. A multi-agent pipeline decomposes generation into specialized sub-tasks. GIR drives stepwise scene expansion through repeated graph read-generate-commit cycles. PGCR validates candidate updates before graph commitment to ensure structural and semantic consistency. Two runtime extensions further enhance the framework: LBP improves branch selection through lightweight rollout evaluation, while JITC dynamically introduces new characters only when required by the evolving narrative.

GRAIL-VN adopts a multi-agent architecture in which all agents share the NWG and interact through GIR and PGCR to generate, validate, and commit candidate updates.

The Planner initializes the narrative world, chapter structure, and character briefs; the Casting Agent instantiates character entities and supports on-demand creation via JITC; the Director expands scene topology and candidate branches; the Actor generates Beat-level content and state transitions; and the Judge performs semantic validation in PGCR and branch evaluation in LBP.

Narrative World Graph

GRAIL-VN models the narrative world as a property graph and uses it as a persistent runtime state interface during generation. Unlike graphs that serve only as retrieval resources or static background knowledge, NWG serves as the persistent runtime state interface throughout generation, providing state retrieval, alignment targets, and state commitment for candidate updates.

NWG mainly contains three groups of core nodes: hierarchical narrative units (**Global**, **Chapter**, **Scene**, and **Beat**), persistent character entities (**Character**), and state snapshots attached to characters (**NarrativeState**). These nodes are not used to completely enumerate narrative fields, but rather serve runtime generation: narrative units organize local scene and Beat updates, Character provides stable en-

tity references, and NarrativeState exposes inspectable state snapshots.

Core relations connect narrative topology and state evolution. **NEXT** and **BRANCH** encode the executable story structure between Scenes, where **BRANCH** stores structured predicates for branch selection. **HAS_CURRENT_STATE** links each Character to its latest visible state snapshot, providing a unified runtime state for Agents and checkers. **UPDATES_STATE** records the state transitions introduced by Beats or scene events, connecting narrative actions with their resulting state changes.

Graph-Backed Iterative Runtime

We define graph-backed iterative runtime as an execution paradigm in which each generation step explicitly reads from and commits updates to a persistent narrative state graph. Unlike conventional generate-once pipelines, the runtime explicitly interleaves generation and state commitment, allowing subsequent steps to operate on updated narrative states.

Specifically, the system maintains a queue of scenes to be generated within a chapter, and in the current implementation expands them in breadth-first order. The Director takes the current Scene from the queue, reads its predecessors, chapter goals, known characters, and current NarrativeState, and generates the content of the scene as well as candidate outgoing edges. If the candidate update passes PGCR, the system writes the Scene, NEXT/BRANCH edges, and related structures back to NWG, and adds the target scenes that need further expansion to the queue. The Actor then reads the latest context of the scene and generates the Beat sequence and state transitions; these Beat-level updates are likewise subject to structural, state, and semantic checks before being written back. After a scene or chapter is completed, the system generates and writes back a compressed summary, enabling subsequent generation to access historical progress in compressed form.

This queue-based expansion process constitutes the core of GIR in the experiments of this paper: even when PGCR, NSM, Judge, and JTC are disabled, the system still operates according to the pattern of “read graph–local generation–write back–continue expansion.” Compared with bulk generation, the key difference of the iterative runtime is that the graph is not merely a one-time context before generation, but rather a transactional interface for passing state between generation steps. LBP reuses this queue-based expansion process, but adds lookahead scoring and pruning only when “the current scene produces multiple candidate outgoing edges.”

PGCR Protocol: Pre-Commit Verification for Candidate Updates

PGCR is the pre-commit protocol used by GRAIL-VN to determine whether a candidate update can be safely committed to NWG. For any generation step t , the constrained write-back process can be abstracted as

$$I_t^* = \text{Constrain}(\text{Align}(\text{Plan}(q_t, G_t), G_t)). \quad (2)$$

$$G_{t+1} = \text{Realize}(I_t^*, G_t). \quad (3)$$

Here, G_t denotes the current runtime state of the Narrative World Graph at generation step t . q_t denotes the local task context of the current generation step, such as the chapter goal, the scene to be expanded, a character brief, or the context for a Beat to be generated. I_t^* denotes the structured intent after alignment and constraint checking. If a constraint fails, the system returns structured feedback and replans within the maximum number of retries $N_{\max}$. PGCR decomposes single-step generation into four stages:

Stage 1: Plan. The Agent generates a structured intent representation based on the current narrative state. Its fields cover the minimal sufficient information required for subsequent validation and write-back, such as event sequences and state transition declarations.

Stage 2: Graph Align. The system maps entity references, state fields, and structural identifiers in the intent to current graph objects, performing entity disambiguation and reference alignment. Items that cannot be aligned are converted into structured feedback to guide subsequent replanning.

Stage 3: Constrains. After graph alignment, the system applies the following three layers of validation to the Intent:

- *Layer 1 — Structural and field-level validation:* checks whether the Intent fields satisfy the structural constraints of the graph, such as `beat_id` uniqueness, valid character names, and valid branch condition fields, and filters structural errors that cannot be written back to the graph.
- *Layer 1b — State-path validation:* checks whether the starting value in every state transition declaration is consistent with the corresponding character’s HAS_CURRENT_STATE snapshot in the graph. To support this check, GRAIL-VN introduces a Narrative State Machine (NSM), which maintains each character’s state chain through NarrativeState snapshots, HAS_CURRENT_STATE pointers, and **TRANSITIONS.TO** edges. This layer prevents the LLM from modifying state based on an incorrect premise. For example, if the model claims to “change affinity from 3 to 5” while the graph records the current value as 7, a violation is triggered and correction is required.
- *Layer 2 — Semantic Judge validation:* when no violation is found in Layer 1 or Layer 1b, the system calls the Judge Agent to perform semantic-level review of the entire Intent in terms of narrative coherence and world-setting consistency, capturing logical contradictions that structural validation cannot detect, such as character behavior that seriously conflicts with the character profile.

If any layer fails, the system produces structured feedback and retries generation up to $N_{\max}$ times.

Stage 4: Realize. After validation passes, the system realizes the structured intent as concrete narrative content and its corresponding graph nodes, relations, and state updates, and commits them to the Narrative World Graph, completing the advancement of nodes, relations, and state. This stage may

also trigger runtime extensions such as online character creation.

PGCR provides a unified runtime interface for detection, rejection, and repair before state commitment. Its behavior is further analyzed through representative runtime cases.

Runtime Extensions: LBP and JITC

On top of the main loop formed by NWG, GIR, and PGCR, GRail-VN provides two lightweight runtime extensions. LBP introduces local lookahead-based filtering when a scene contains multiple candidate branches, while JITC expands the character set on demand during entity alignment and commitment.

LBP is triggered when the current scene yields multiple structurally valid outgoing branches. Let $\mathcal{B}_{\text{valid}}$ denote the candidate set retained after PGCR structural checking, and let $J(b) = s_{\text{consistency}}(b) + s_{\text{drama}}(b) + s_{\text{logic}}(b)$ denote the Judge score for branch b , covering consistency, dramatic potential, and logical coherence. LBP performs a short-horizon rollout for each candidate branch, summarizes the resulting trajectory, and evaluates the rollout summary with the Judge to obtain $J(b)$. It then keeps the Top- K branches:

$$\mathcal{B}_{\text{keep}} = \text{TopK}_{b \in \mathcal{B}_{\text{valid}}} J(b). \quad (4)$$

LBP therefore ranks and filters candidate outgoing edges according to their short-horizon narrative consequences before they are committed and expanded further. It focuses on the relative quality of local candidates and provides the Director with a more stable basis for branch selection.

JITC addresses open-ended character introduction in branching narratives. When enumerating all potential entities in advance is impractical, JITC checks the characters declared by the current scene during scene expansion, entity alignment, and the Realize stage. For a scene node v , let $C_{\text{present}}(v)$ be the set of characters present in the scene and C_{global} the current global cast already committed to NWG. The set of new character candidates is

$$\Delta C(v) = C_{\text{present}}(v) \setminus C_{\text{global}}. \quad (5)$$

If $\Delta C(v)$ is nonempty, the system first performs entity disambiguation and attempts to reuse existing characters. Only unresolved candidates are instantiated by the Casting Agent under the current narrative constraints and committed to the NWG with their profiles and initial states. Once committed, the new characters become available for later graph retrieval, PGCR validation, and scene generation. This reuse-before-creation policy supports dynamic expansion of the narrative world while reducing duplicate characters and one-off entity fragmentation.

Experiments

This section organizes the evaluation around three progressively layered research claims. First, we examine whether the graph-backed runtime loop, consisting of the Narrative World Graph, iterative runtime, and PGCR, can improve the structural validity and state consistency of branching visual-novel scripts. Second, we evaluate whether LBP, as a branch-selection hook during scene expansion, can improve local

planning and filtering of candidate branches. Finally, we test whether JITC, as a character-expansion mechanism during entity alignment and write-back, can expand the character set in a controlled and reusable manner. To this end, we combine automated structural metrics, LLM-as-Judge comparisons of narrative quality, and goal-achievement evaluation to form a multi-level chain of evidence spanning runtime executability, mechanism diagnostics, and external quality validation.

Setup

We constructed an evaluation set containing 36 Briefs, covering four genres (Romance, Mystery, Horror, and Fantasy) and three complexity levels (S/M/C). The S category typically contains 2–3 characters and 0–1 branch points, the M category contains 3–4 characters and 1–2 branch points, and the C category contains 4 characters and 3 branch points while allowing multiple endings. Each genre–complexity combination contains 3 content-distinct Briefs, for a total of 36 samples; each Brief explicitly specifies 3–5 assessable narrative goals. This genre $\times$ complexity design ensures that the evaluation is not limited to a single narrative genre or a single level of branching complexity. Briefs were generated using a fixed prompting procedure and manually screened to ensure genre, complexity, and goal coverage. They are provided as natural-language narrative specifications in English and are released in the code repository.

All generation configurations use the same backbone generation model, DeepSeek-v4-Flash (DeepSeek-AI 2026), and set scene/beat budgets by complexity: 3/4 for S, 4/5 for M, and 5/6 for C. To control randomness, all runs use fixed random seeds. The external LLM-as-Judge and Goal Achievement evaluations consistently use Qwen3.5-Flash (Qwen Team 2026), separated from the generation model to reduce same-model evaluation bias. The experiments in this paper evaluate only the main textual narrative pipeline and do not include optional downstream modules such as Artist/Musician, because the research questions of this paper focus on structural generation, consistency control, and branch decision-making.

Metrics

The evaluation metrics are organized by type of evidence. We first use automated structural and state metrics to characterize the runtime reliability of the graph-backed iterative runtime; we then use PGCR trace metrics to examine Detect–Retry–Repair capability; next, we use LBP/JITC diagnostic metrics to analyze the local behavior of the two extension mechanisms; finally, we use external LLM-as-Judge and Goal Achievement as boundary validation to assess whether these runtime mechanisms correspond to perceptible quality or brief-level goal fulfillment.

- **Structural and state outcome metrics:** Compilation Rate (CR) measures whether a script can be successfully compiled and pass structural validation. Logic Bug Rate (LBR) is defined as the ratio of structural script errors to all structural checkpoints, mainly covering automatically

Configuration	Read Graph	Text Memory	PGCR-L1	PGCR-L1b	PGCR-L2	LBP	JITC	Mode
Bulk	N	N	N	N	N	N	N	Bulk
Graph-Bulk	Y	N	N	N	N	N	N	Bulk
TMI	N	Y	N	N	N	N	N	Iterative
GIR	Y	N	N	N	N	N	N	Iterative
GIR+L1	Y	N	Y	N	N	N	N	Iterative
GIR+NSM	Y	N	Y	Y	N	N	N	Iterative
GIR+Judge	Y	N	Y	Y	Y	N	N	Iterative
GIR+JITC	Y	N	Y	Y	Y	N	Y	Iterative
LBP	Y	N	Y	Y	Y	Y	N	Iterative
LBP+JITC	Y	N	Y	Y	Y	Y	Y	Iterative

Table 1: Evaluation configuration matrix.

detectable topological errors such as empty scenes, non-terminal scenes without outgoing edges, invalid transition references, and BRANCH nodes missing conditions. State Transition Validity Rate (STVR) measures the proportion of declared state transitions whose previous state is consistent with the current NarrativeState snapshot, and is used to characterize the validity of local state paths after NSM is enabled.

- **PGCR capability metrics:** PGCR traces are defined only for configurations with PGCR enabled. We report CPR@1 (candidate pass rate at the first attempt, i.e., the proportion of candidates that directly pass all PGCR checks on their first submission), Final Pass (the proportion of candidates that eventually pass and are successfully written back within the maximum number of retries), Repair SR (the proportion of initially failed candidates that are ultimately repaired successfully after feedback and retry), and Avg Attempts (the average number of attempts before a candidate either finally passes or is abandoned), as well as layer-wise violation rates calculated by the actual number of executed checks. These metrics characterize admission, rejection, and repair behavior before candidate write-back. Non-PGCR baselines do not produce these traces.
- **Auxiliary mechanism metrics:** For JITC, we report JTR (the proportion of scenes that trigger JITC character expansion), JCIR (the proportion of JITC-related scenes that reuse already created characters), Reuse Char Rate (the proportion of newly created characters that are reused at least once later), and One-Scene-Only (the proportion of new characters that appear in only a single scene). For LBP, we report Keep-Prune Gap (the average score difference between retained and pruned branches) and Cutoff Margin (the average score gap at the retain/prune boundary), which characterize the internal diagnostic signals of branch filtering.
- **External validation metrics:** We use an external LLM-as-Judge evaluator that differs from the generation backbone, conducting pairwise comparisons across six dimensions: Plot, Character Fidelity, State Consistency, Narrative Coherence, Branch Quality, and Overall. To reduce positional bias, each comparison uses AB/BA dual-order evaluation, and the order-aggregated win/draw/lose

results serve as the main Judge output. For Goal Achievement (GA), we score the preset goals of each Brief item by item on a 0/0.5/1 scale and average the goal scores within the same Brief.

Evaluation Configurations

Beyond the progressive internal configurations of the framework, this paper also introduces controlled baselines to compare GRAIL-VN against common LLM narrative-generation paradigms.

Bulk is a prompt-chain style bulk baseline: the system still first generates the world setting, characters, and chapter plan, but during script generation it does not perform generation-time graph reads, PGCR, NSM, Judge, or JITC, and instead plans the scenes and Beats within each chapter all at once in bulk mode. Graph-Bulk is a static graph-context baseline: it reads the initialized NWG during bulk generation, but does not perform iterative graph write-back, state commits, or PGCR checks between scenes or Beats. This configuration is used to distinguish “one-time graph context” from “a graph that continuously participates in generation as a runtime state interface.” TMI (Text-Memory Iterative) is a strong textual-memory baseline: it uses the same iterative budget as GIR, but disables graph reads and all pre-commit constraints. The system maintains only pure-text rolling memory, using completed chapter summaries as story progress for the subsequent Director and recent scene summaries plus key Beat texts as recent events for the subsequent Actor. All configurations share a unified graph-writing, compilation, and automatic-evaluation base to ensure that output structures and metric definitions are comparable; the main differences lie in whether structured graph state is read during generation, whether pure-text memory is maintained, and whether pre-commit constraints and extension mechanisms are enabled.

Results and Analysis

Before the detailed tables, we summarize the key takeaways:

- The graph’s value comes from its continuous participation as a runtime state interface, not from one-time context provision.

- PGCR forms an observable detect–retry–repair path before candidate write-back, repairing most first-round failures within limited retries.
- LBP and JIRC add branch-selection and entity-expansion mechanisms to the same loop, showing clear diagnostic signals and positive but limited external evidence.

Main Results

This section jointly reports final script-level outcome metrics and runtime-intervention metrics from the generation process, so that the final output performance of each configuration and its pre-write-back PGCR behavior can be compared under a unified view. Table 2 summarizes structural and state outcomes, candidate-level admission/repair metrics, and layer-wise diagnostic rates; Table 3 diagnoses the local behavior of the two runtime hooks, LBP and JIRC. Unless otherwise specified, all results are based on the same set of 36 Briefs.

CR is 100% across all configurations and is therefore not listed separately in the main table.

For final outcomes, static graph context alone brings little gain: Bulk and Graph-Bulk reach LBR 0.0047 and 0.0048, whereas GIR reduces LBR to 0.0002, confirming that the NWG’s value lies in continuous runtime participation.

TMI further shows that iterative textual memory is already a strong final-output baseline: without reading the structured graph and without executing PGCR, it still achieves a low LBR (0.0005). This result indicates that final shallow structural metrics alone are insufficient to fully characterize the differences between the two types of methods. Compared with textual-memory-based iterative generation, the main gains of GRAIL-VN lie in runtime state control capability, specifically including explicit state commits, candidate-level admission checks, and auditable repair traces.

For state-path-related metrics, configurations with NSM enabled all maintain high STVR (approximately 0.97–0.98), indicating that explicit state-path checking can maintain high local state-transition validity under richer stateful generation.

For candidate-level admission, repair, and layer-wise diagnostic metrics, Table 2 more directly demonstrates the runtime role of PGCR. The CPR@1, Final Pass, and Repair SR of GIR+L1 reach 0.9835, 0.9995, and 0.9669, respectively. After the Judge layer is introduced, CPR@1 drops to 0.8846, but Final Pass remains at 0.9866 and Repair SR reaches 0.8836. This indicates that stronger constraints substantially reduce the first-pass rate of candidates, but most candidates rejected on the first attempt can still be repaired after feedback and retry and then successfully written back. At the same time, Avg Attempts consistently remains close to 1.0–1.16, indicating that most candidates can be repaired within limited retries and that the overall cost of runtime intervention is controllable. The layer-wise diagnostic rates further indicate that, with the introduction of state-path checking and the semantic Judge, candidate-level intervention gradually expands from basic structural constraints to state consistency and semantic consistency. Taken together, these metrics suggest that the main role of PGCR is to move

problem handling before candidate write-back and to control runtime state updates through admission, rejection, and repair mechanisms.

Table 3 reports the diagnostic results for the LBP and JIRC extension mechanisms. The JIRC-related metrics show that, although this mechanism is triggered at a relatively low frequency (JTR of approximately 0.07), new characters usually continue to participate in subsequent narrative after being introduced. The high JCIR (approximately 0.84) indicates that, among JIRC-related scenes, a large share reuse already created characters; the Reuse Char Rate of approximately 0.74–0.79 further indicates that a high proportion of newly created characters appear again. Meanwhile, One-Scene-Only is approximately 0.21–0.26, indicating that one-off characters still exist but do not dominate. Overall, these results support JIRC as a low-frequency but effective online character-expansion mechanism. For LBP, the Keep-Prune Gap is approximately 8.8 and the Cutoff Margin is approximately 5–6, indicating that lookahead scoring not only forms a stable gap between retained and pruned branches, but also preserves an observable decision margin near the Top-K retention boundary. The following sections complement these aggregate statistics with representative PGCR repair cases and external quality evaluation.

PGCR Case Analysis: Representative Failure Patterns and Repair

Aggregate statistics cannot show how each constraint layer intervenes within a single generation step; Table 4 therefore extracts representative failure patterns and their repair paths from runtime logs.

Table 4 shows that the three PGCR checking layers do not intercept the same type of error: L1 mainly handles the validity of controlled fields, L1b constrains the consistency between candidate state paths and the current graph snapshot, and L2 further examines coherence among scene semantics, emotional progression, and relationship evolution. Together with the aggregate results in Table 2, these log-level cases further show that the role of PGCR is not only to reject unqualified candidates, but also to provide targeted repair feedback and repair a substantial portion of candidates into a committable state.

External Quality and Goal Preservation

One notable characteristic of Table 5 is the relatively high draw rate across all comparison groups, indicating that the reader-facing differences among most configurations are generally moderate. Although the overall differences are limited, the table still shows several relatively stable local positive signals. Against the strong TMI baseline, LBP+JIRC achieves win/draw/lose ratios of 38.9/30.6/30.6, 36.1/44.4/19.4, and 30.6/47.2/22.2 on Plot, Character Fidelity, and Narrative Coherence, respectively. This indicates that GRAIL-VN exhibits a more frequent win-than-lose pattern at least on dimensions that are more susceptible to runtime state control, such as plot progression, character consistency, and cross-scene continuity.

The internal comparisons help clarify which runtime mechanisms these local differences may correspond to.

Configuration	Final Outcomes		PGCR Admission / Repair				Layer-wise Diagnostics		
	LBR	STVR	CPR@1	Final Pass	Repair SR	Avg Attempts	L1 VR	L1b VR	L2 VR
Bulk	0.0047	—	—	—	—	—	—	—	—
Graph-Bulk	0.0048	—	—	—	—	—	—	—	—
TMI	0.0005	—	—	—	—	—	—	—	—
GIR	0.0002	—	—	—	—	—	—	—	—
GIR+L1	0.0002	—	0.9835	0.9995	0.9669	1.0188	1.95%	—	—
GIR+NSM	0.0012	0.9785	0.9760	0.9983	0.9293	1.0277	10.38%	1.52%	—
GIR+Judge	0.0016	0.9777	0.8846	0.9866	0.8836	1.1478	9.48%	17.32%	16.28%
GIR+JITC	0.0004	0.9775	0.8787	0.9838	0.8667	1.1551	10.14%	18.56%	16.79%
LBP	0.0009	0.9741	0.8725	0.9848	0.8811	1.1586	11.68%	18.60%	16.97%
LBP+JITC	0.0009	0.9750	0.8931	0.9890	0.8975	1.1316	7.88%	15.56%	13.95%

Table 2: Main results and PGCR runtime diagnostics. L1/L1b/L2 violation rates are runtime diagnostic rates computed over executed checks, not final script error rates.

Configuration	JTR	JCIR	Reuse Char Rate	One-Scene-Only	Keep-Prune Gap	Cutoff Margin
GIR+JITC	0.0712	0.8462	0.7857	0.2143	—	—
LBP	—	—	—	—	8.8448	5.2388
LBP+JITC	0.0676	0.8441	0.7385	0.2615	8.7669	5.7686

Table 3: Runtime extension diagnostics.

Relative to GIR, GIR+JITC reaches 36.1/41.7/22.2 and 36.1/47.2/16.7 on State Consistency and Branch Quality, respectively, indicating that online character expansion does not undermine overall narrative stability and shows positive signals in state consistency and branch development quality. Further, relative to GIR+JITC, LBP+JITC reaches 36.1/41.7/22.2 on Branch Quality and 36.1/44.4/19.4 on Overall, indicating that the benefits of lookahead branch selection are concentrated first in the Branch Quality dimension and may further spill over into overall preference. The relatively stable Keep-Prune Gap and Cutoff Margin in Table 3 also support this interpretation: at runtime, LBP can continuously widen the score gap between retained candidates and pruned candidates, and this internal selection difference subsequently appears in Table 5 as better Branch Quality results. At the same time, LBP+JITC does not form a corresponding advantage over GIR+JITC on State Consistency, which also indicates that the benefits of LBP are more concentrated in branch organization and selection quality rather than producing consistent improvements across all external dimensions.

Table 6 supplements the above Judge results from the perspective of brief-level goal coverage. Bulk reaches only 48.8%, indicating that one-shot prompt-chain generation has difficulty stably covering the narrative goals explicitly specified in the brief. By contrast, TMI and LBP+JITC reach 78.3% and 77.9%, respectively; both are clearly higher than Bulk and are essentially comparable to each other. This result indicates that, after introducing runtime graph control, PGCR, JITC, and LBP, GRAIL-VN does not obtain structural control capacity at the expense of goal achievement. Taken together with Table 5, the more cautious conclusion is that these runtime mechanisms remain competitive with a strong iterative text baseline in external evaluation while

showing local positive signals on several structure-sensitive dimensions.

Taken together, these results indicate that the primary benefits of GRAIL-VN still lie in runtime state control, candidate-level repair, and auditable write-back processes. Compared with a strong text-memory baseline, these capabilities are not merely internal mechanistic differences; when the generation target is a stateful branching narrative object that requires subsequent execution, expansion, and constrained commitment, this runtime control framework is also easier to inspect, repair, and maintain.

Conclusion

This paper presents GRAIL-VN, a runtime control framework for branching visual novel script generation. Through NWG, GIR, and PGCR, the system organizes branching narrative generation as a series of candidate state updates that can be inspected, rejected, repaired, and committed; LBP and JITC further extend the system’s capabilities for branch selection and online entity expansion.

The experimental results can be summarized in three points. First, the main value of the graph comes from its continuous participation as a runtime state interface rather than from one-time context provision. Second, PGCR forms an observable Detect-Retry-Repair path before candidate write-back, allowing a considerable portion of first-round failed candidates to be repaired after feedback and re-enter the commitment process. Third, LBP and JITC show that additional runtime mechanisms can be embedded into the same control loop; on the one hand, they exhibit clear mechanistic benefits in runtime diagnostics, and on the other hand, they also show positive results on some external dimensions.

Layer	Representative Failure Type	Typical Violation Manifestation	Common Repair Method
L1	Invalid enumeration value or unregistered field value	Candidate branches repeatedly generate values for <code>emotional.state</code> that are not in the controlled vocabulary, such as <code>resigned</code> , <code>uncertain</code> , <code>resolute</code> , and <code>self.doubt</code>	Normalize the candidate value to the valid vocabulary, or feed back to the model to regenerate a controlled value
L1b	State-path starting value inconsistent with the current graph value	The candidate-declared <code>from.value</code> does not match <code>graph.current</code> ; for example, Helena’s current emotion is <code>Hope</code> , but the candidate starts from <code>Anxiety</code>	Align the starting state with the current graph value and resubmit while preserving the target state
L2	Conflict between emotional arc and state transition	The start and end states of <code>emotional.direction</code> and <code>state.transition</code> conflict, or the scene summary already indicates that the character is <code>sad/lonely</code> , yet the candidate still starts from <code>calm</code>	Synchronously adjust <code>from.value</code> , <code>to.value</code> , and <code>emotional.direction</code> ; if necessary, reorder Beats or rewrite the local emotional progression
L2	Relationship stage inconsistent with story history	The candidate advances the Helena–Dani relationship from <code>Stranger</code> to <code>Partner</code> , but previous scenes already show that the two have established cooperation and mutual trust	Adjust the relationship starting value back to a stage consistent with the existing story, such as <code>Acquaintance</code> , <code>Friend</code> , or <code>Trusted Partner</code>

Table 4: Representative PGCR failure patterns and repair paths from runtime logs.

Comparison (A vs. B)	Plot	Char Fid.	State Cons.	Narr. Coh.	Branch Qual.	Overall
LBP+JITC vs. TMI	38.9/30.6/30.6	36.1/44.4/19.4	30.6/41.7/27.8	30.6/47.2/22.2	27.8/44.4/27.8	30.6/38.9/30.6
GIR+JITC vs. GIR	30.6/33.3/36.1	22.2/58.3/19.4	36.1/41.7/22.2	27.8/47.2/25.0	36.1/47.2/16.7	25.0/47.2/27.8
LBP+JITC vs. GIR	33.3/52.8/13.9	27.8/50.0/22.2	33.3/30.6/36.1	27.8/47.2/25.0	33.3/47.2/19.4	33.3/38.9/27.8
LBP+JITC vs. GIR+JITC	30.6/33.3/36.1	30.6/36.1/33.3	27.8/38.9/33.3	27.8/52.8/19.4	36.1/41.7/22.2	36.1/44.4/19.4

Table 5: Pairwise narrative quality comparison (Win / Draw / Lose, N=36; higher Win and lower Lose are better).

Configuration	GA
Bulk	48.8%
TMI	78.3%
LBP+JITC	77.9%

Table 6: Goal Achievement sanity check (higher is better).

Limitations This work still has three limitations. In the state management mechanism, the current implementation maintains a state snapshot only on a single active narrative path; “branching” here refers to the branching narrative structure produced during incremental expansion, not to the simultaneous maintenance of independent state snapshots over diverged branches. Parallel, independent multi-branch snapshot management, and runtime decisions on whether to continue expanding a branch, remain future work. In the evaluation method, narrative quality evaluation still partially depends on LLM-as-Judge, and the intervention and repair statistics for PGCR also mainly come from internal system traces; these cannot yet replace human evaluation and larger-

scale external semantic audits. In the depth of validating extension mechanisms, LBP implements only local one-step lookahead, JITC is validated only on character entities, and stronger control baselines are still lacking.

Future Works Based on the above limitations, future work will proceed mainly in three directions. First, `NarrativeState` will be extended into a snapshot tree indexed by branch path, with further support for conditional evaluation based on state snapshots and runtime control that decides whether a branch should continue to expand according to the current state. Second, runtime extension mechanisms will be improved, including multi-step LBP, a branch scoring mechanism that uses lookahead results to update candidate priorities in reverse, and JITC extensions for more narrative entity types such as props and locations. Finally, stricter external evaluation will be added, and the generalizability of the conclusions in this paper will be validated on larger-scale and more diverse narrative briefs.

Acknowledgments

We thank Prof. Mingming Liu for her guidance and support, and Kaijie Xu for his helpful discussions and feedback.

References

- Alabdulkarim, A.; Li, S.; and Peng, X. 2021. Automatic story generation: Challenges and attempts. In *Proceedings of the Third Workshop on Narrative Understanding*, 72–83.
- Chen, J.; Zhu, X.; Yang, C.; Shi, C.; Xi, Y.; Zhang, Y.; Wang, J.; Pu, J.; Feng, T.; Yang, Y.; et al. 2024. HoLLMwood: Unleashing the creativity of large language models in screenwriting via role playing. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 8075–8121.
- DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348.
- Gardent, C.; Shimorina, A.; Narayan, S.; and Perez-Beltrachini, L. 2017. The WebNLG challenge: Generating text from RDF data. In *Proceedings of the 10th international conference on natural language generation*, 124–133.
- Guan, J.; Huang, F.; Zhao, Z.; Zhu, X.; and Huang, M. 2020. A knowledge-enhanced pretraining model for commonsense story generation. *Transactions of the Association for Computational Linguistics*, 8: 93–108.
- Han, S.; Chen, L.; Lin, L.-M.; Xu, Z.; and Yu, K. 2024. IB-SEN: Director-actor agent collaboration for controllable and interactive drama script generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1607–1619.
- Kim, H.; Yoo, T.; and Cheong, Y.-G. 2025. CoDi: A Director-Actor Framework for Goal-Driven Interactive Story Generation with LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 21, 70–80.
- Koncel-Kedziorski, R.; Bekal, D.; Luan, Y.; Lapata, M.; and Hajishirzi, H. 2019. Text generation from knowledge graphs with graph transformers. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2284–2293.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhume, S.; Yang, Y.; et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36: 46534–46594.
- Meehan, J. R. 1977. TALE-SPIN, An Interactive Program that Writes Stories. In *IJCAI*, volume 77, 91–98.
- Mirowski, P.; Mathewson, K. W.; Pittman, J.; and Evans, R. 2023. Co-writing screenplays and theatre scripts with language models: Evaluation by industry professionals. In *Proceedings of the 2023 CHI conference on human factors in computing systems*, 1–34.
- Park, J. S.; O’Brien, J.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, 1–22.
- Qwen Team. 2026. Qwen3.5: Accelerating Productivity with Native Multimodal Agents.
- Riedl, M. O.; and Young, R. M. 2010. Narrative planning: Balancing plot and character. *Journal of Artificial Intelligence Research*, 39: 217–268.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Wang, Y.; Zhou, Q.; and Ledo, D. 2024. StoryVerse: Towards co-authoring dynamic plot with LLM-based character simulation via narrative planning. In *Proceedings of the 19th International Conference on the Foundations of Digital Games*, 1–4.
- Ware, S. G.; and Siler, C. 2021. Sabre: A narrative planner supporting intention and deep theory of mind. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 17, 99–106.
- Yang, K.; Klein, D.; Peng, N.; and Tian, Y. 2023. Doc: Improving long story coherence with detailed outline control. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 3378–3465.
- Yao, L.; Peng, N.; Weischedel, R.; Knight, K.; Zhao, D.; and Yan, R. 2019. Plan-and-write: Towards better automatic storytelling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, 7378–7385.
- Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.; et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in neural information processing systems*, 36: 46595–46623.